miniFlink: Declarative Stream Processing Semantics in OCaml

MINIFLINK PROJECT, Independent, Russia

We present miniFlink, a functional implementation of Apache Flink semantics in OCaml that demonstrates how declarative stream processing reduces accidental complexity. The key insight is the `KEYED` type class: by encoding the grouping key once in the type, operators `window`, `enrich`, and `dedup` require no explicit `~key:` parameter, making the pipeline read as a specification rather than an implementation. We implement event time, watermarks (with periodic emission, end-of-stream flush, and wall-clock idle advancement), windowing in five flavors (tumbling, sliding, count, session with merging, and a global window with custom triggers) with both materializing and incremental ($O(1)$-memory) aggregation plus a library of composable aggregators, stateful operators, exactly-once checkpointing (single-thread and parallel via Chandy-Lamport barrier snapshots, with source-offset recovery and a transactional sink), table and join semantics with TTL eviction, retractions for late data, and production layers (dead letter queue with retry/backoff, graceful shutdown, Prometheus metrics, structured logging, health and config as values, RocksDB-backed persistent state via C FFI) in approximately 3000 lines, validated by 27 test suites including composition-invariant property tests, replay-determinism, fuzzing, and soak tests. A parallel backend supports both OCaml 4 (Thread + Mutex) and OCaml 5 (Domain + Atomic) with measured speedup of 2.48× on 4 workers (1 CPU).

ACM Reference Format:
miniFlink Project. 2026. miniFlink: Declarative Stream Processing Semantics in OCaml. 1, 1 (June 2026), ?? pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 DECLARATIVITY AS A FIRST PRINCIPLE

Traditional stream processing frameworks require the programmer to explicitly state how to group events, how to serialize them, and how to handle null values at every operator. Consider the same pipeline in Kafka Streams (Java) and miniFlink (OCaml):

Listing 1. Kafka Streams (Java) — imperative

```
KStream<String, Telemetry> stream =
    builder.stream("vehicles.telemetry");
KGroupedStream<String, Telemetry> grouped = stream
    .filter((k, v) -> v != null)
    .groupBy((k, v) -> v.getDeviceId(),
             Grouped.with(Serdes.String(), telemetrySerde));
KTable<Windowed<String>, Stats> stats = grouped
    .windowedBy(TimeWindows.ofSizeWithNoGrace(
                    Duration.ofSeconds(30)))
```

Author's address: miniFlink Project, Independent, Russia.

ACM XXXX-XXXX/2026/6-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

```
    .aggregate(Stats::empty,
               (k, v, acc) -> acc.add(v),
               Materialized.as("stats-store"));
stats.toStream()
    .flatMapValues(Rules::check)
    .filter((k, v) -> v != null)
    .to("fleet.alerts");
```

Listing 2. miniFlink (OCaml) — declarative

```
let pipeline source =
  source
  |> Event.with_watermarks   ~latency:(seconds 3)
  |> Pipe.enrich (module Telemetry)
       ~from:devices
       ~merge:(fun t dev -> { t with device = dev })
  |> Pipe.window (module Telemetry) (tumbling (seconds 30))
  |> Pipe.aggregate              Rules.compute
  |> Pipe.flat_map               (Rules.check Rules.fleet)
  |> Pipe.dedup (module Alert)
       ~rule:(fun a -> a.rule)
       ~cooldown:(minutes 5)
```

The differences are structural, not cosmetic:

- **No `key_by`.** The `KEYED` type class encodes the grouping key in the type once; `window`, `enrich`, and `dedup` extract it automatically.
- **No type annotations** inside the `|>` chain. Hindley-Milner inference propagates types through all operators.
- **No null checks.** `Option` makes absence explicit; `None` cannot be dereferenced.
- **Format in one place.** Changing JSON to Protobuf means replacing one `Codec` value; the pipeline is untouched.

2 ARCHITECTURE

miniFlink is organized into four orthogonal layers. Each layer depends only on layers below it; the core never imports reliability or parallelism modules.

Table 1. Layer organization

Layer	Modules	Responsibility
Core	`stream, event, pipe, keyed, table, codec`	Immutable. No external dependencie
Domain	`domain, rules`	`[@@deriving yojson]`. Pure functio
Parallelism	`channel_v4/v5, parallel_v4/v5`	OCaml 4: Thread+Mutex. OCaml 5
Reliability	`dlq, shutdown, metrics, barrier, schema`	Each: `.mli` + `_noop` + `_default`.

The `Makefile` detects `OCAML_MAJOR` and copies the appropriate `channel_v{4,5}.ml` and `parallel_v{4,5}.ml` before compilation.

3 CORE ABSTRACTIONS

3.1 Pull Stream

```
type 'a t = unit -> 'a option
```

A stream is a function. The call stack *is* the operator graph. No callbacks, no threads, no allocation per event beyond the value itself. `map`, `filter`, `flat_map` compose without intermediate buffers.

3.2 Event

```
type 'a t =
  | Data      of 'a * timestamp    (* value + event time *)
  | Watermark of timestamp         (* lower bound on future ts *)
  | Retract   of 'a * timestamp    (* revocation of prior Data *)
```

All three constructors flow through the same stream. Operators match exhaustively — adding a constructor forces all operators to handle it at compile time.

3.3 KEYED Type Class

```
module type S = sig
  type t
  val key : t -> string
end

(* Registered once per domain type *)
module Telemetry = Keyed.Make(struct
  type t = telemetry
  let key t = t.device_id
end)
```

Operators receive `(module Telemetry)` as a first-class module and call `K.key` internally. The user never writes `~key:(fun t -> t.device_id)` again.

3.4 Window

The locally abstract type annotation breaks the occurs-check that would otherwise create the recursive type $\alpha = \texttt{string} \times \alpha\ \texttt{list}$:

```
let window (type a) ?(latency=0)
    ~(key : a -> string) spec
    (upstream : a Event.t Stream.t)
    : (string * a list) Event.t Stream.t = ...
```

Window state is a `WMap.t` from `(key, start, stop)` triples to event lists. Watermarks trigger partition of open windows into *closed* (fired) and *open* sets; closed windows emit their accumulated events downstream.

Beyond time-based `tumbling` and `sliding`, miniFlink offers three more window types as separate operators. **Count windows** fire by event count rather than time, needing no watermarks. **Session windows** have data-driven boundaries: a session grows while events arrive within `gap` and *merges* with a neighbor when an event bridges the gap between two sessions — this breaks the "window is a pure function of timestamp" assumption the time-based windows rely on, so it carries its own session state and merge logic. **Global**

windows hold one window per key and delegate the fire decision to a pluggable *trigger* (`Continue` | `Fire` | `FireAndPurge`), separating the assigner (how to group) from the trigger (when to emit) — the foundation other firing policies build on.

Aggregation over windows comes in two forms. The materializing path (`window |> aggregate`) keeps the full event list and folds it post-hoc — $O(n)$ memory per window. The incremental path (`window_fold ~init ~add`) folds each event into an accumulator on arrival — $O(1)$ memory in the event count. On top sits a small library of composable aggregators (`Agg`): `count`, `sum`, `mean`, `min_by`/`max_by`, `arg_min`/`arg_max`, etc. An aggregator hides its accumulator type behind a GADT existential, so aggregators with different accumulators combine uniformly; `both` (and an applicative `let+`/`and+`) compute several aggregates in a single pass. Feeding such an aggregator to a window (`window_agg`) requires applying the hidden accumulator inside `window_fold`; this is done with a rank-2 continuation (a record with a universally-quantified field) so the existential accumulator type does not escape its scope — a small but genuine use of the GADT/rank-2 combination, not decoration.

4 EVENT TIME AND WATERMARKS

Each event carries its origin timestamp independently of processing time. `with_watermarks_ext ~latency ?interval` inserts `Watermark(max_seen - latency)`. The invariant:

$$\forall t < \texttt{wm}: \text{ no future } \texttt{Data}(\_, t) \text{ will arrive}$$

This allows deterministic window closure without wall-clock polling. Watermarks are monotone by construction — `max_seen` is a `ref` that only grows.

Two refinements over naive per-event emission. **Periodic emission:** the optional `~interval` parameter emits a watermark only once it has advanced by at least `interval` in event-time, collapsing the one-watermark-per-event flood that a monotone stream would otherwise produce. **Final flush:** on stream end the operator emits a terminal watermark equal to `max_seen` (without subtracting latency), guaranteeing every trailing window closes rather than lingering open. A subtle integer overflow — `wm - last_wm` with `last_wm = min_int` — is special-cased so the first watermark is never suppressed.

5 EXACTLY-ONCE CHECKPOINTING

Single-thread. A checkpoint is a triple $(epoch, offset, state)$. The runner reads N events, and on failure restores state from the latest checkpoint, seeks the source to `offset`, and resumes. The key invariant is *ordered commit*: flush sink before writing the checkpoint; write the checkpoint before committing the source offset. Violated ordering creates either duplicates (checkpoint before flush) or gaps (offset before checkpoint).

Parallel (Chandy-Lamport). For data-parallel execution, `Checkpoint_parallel` integrates barriers into the worker channels. Channel messages are wrapped as `Event of _ | Barrier of epoch`. The dispatcher injects a `Barrier(epoch)` into every worker channel every `checkpoint_every` events. On receiving a barrier each worker snapshots its own `state_backend` and submits $(worker, epoch, bytes)$ to the coordinator. The coordinator commits `checkpoint`$(epoch)$ — the array of per-worker snapshots — atomically once *all* workers have reported (barrier alignment; trivial here since each worker has a single input channel). Because key-sharding routes each key to exactly one worker, a committed checkpoint is a consistent cut with no cross-worker double-counting.

The offset-consistency pitfall. The source offset stored in a checkpoint must be *consistent with the snapshotted state*. The obvious implementation — record the dispatcher's

source position at barrier injection — is subtly wrong: the dispatcher reads ahead of what workers have processed, so it may record `offset=2000` while the snapshots reflect only 1900 processed events (the other 100 still in flight in channels). On recovery the source would seek to 2000 and those 100 events would be lost — neither in state nor replayed. The fix: the offset is the *sum of per-worker processed counts captured at the barrier*, taken in the same instant as each snapshot. Offset and state are then consistent by construction; the discrepancy is physically impossible. This bug passed functional tests and surfaced only under a consistency check (offset must equal restored state count) — a concrete illustration of why exactly-once demands invariant testing, not just "does it work" testing.

Transactional output. State-side exactly-once is not enough: replaying the tail after recovery re-runs events through the sink. miniFlink offers two paths. *Idempotent*: results carry a deterministic key and the sink upserts — a replay overwrites with the same value, harmless, no transactions needed (ideal when the output is "state" rather than an event log). *Two-phase commit*: results are written to a per-epoch pre-commit buffer, made visible by `ts_commit` only when that epoch's checkpoint commits, and rolled back by `ts_abort` on failure — bound to the same barrier that commits state. 2PC fits only sinks that separate write from visibility (Kafka transactions, RDBMS, file staging+rename); for HTTP or notifications, idempotency is the only option. A final `ts_flush` publishes the post-last-barrier tail at clean shutdown.

Together — consistent snapshot, offset = Σ processed, durable checkpoint store (atomic `rename` of a `LATEST` pointer), recovery that seeks the source, and a transactional/idempotent sink — these close end-to-end exactly-once on a single node. A dedicated deep-dive is in `docs/exactly_once.md`. Coordinating a real external source/sink (Kafka `transactional.id`) and multi-input barrier alignment remain out of scope.

6 TABLE AND JOIN

```
type ('k,'v) table = 'k -> 'v option

let of_list pairs = Hashtbl.find_opt (Hashtbl.of_seq ...)
let of_stream ~key src = fun k -> pump src; Hashtbl.find_opt h k
```

A table is a function. `Snapshot` tables call `pump` on each lookup to drain pending updates from an upstream stream. `lookup_join` is a one-liner:

```
let lookup_join (module K) ~from ~merge upstream =
  emap (fun v -> merge v (from (K.key v))) upstream
```

`interval_join` buffers both streams; events match when keys are equal and $|t_A - t_B| \leq$ `window_ms`. The combined watermark is $\min($`wm`$_A$, `wm`$_B)$.

`of_stream` is bounded by the number of *distinct* keys (it uses `replace`, not `add`), but an unbounded key space would still grow it without limit. For that case `of_stream_ttl ~key ~ttl` stores $(value, last\_ts)$ per key, tracks the maximum event-time seen, and evicts any key whose $last\_ts <$ `max_ts` $- ttl$ on each pump — the same watermark-driven eviction used by `dedup`, bounding table size under a growing key space.

6.1 Temporal (As-Of) Join

The snapshot/live tables above answer "what is the current value of the dimension". A *temporal* join answers "what was the value *as of* the row's event-time" — the Flink `FOR`

`SYSTEM_TIME AS OF` semantics. The dimension is a *versioned* table: per key, a history of $(valid\_from, value)$ kept sorted, so a late update with an old `valid_from` is inserted in its correct place. `as_of k ts` returns the version with the largest $valid\_from \leq ts$.

```
let temporal_join ~key_main ~key_upd ~valid_from ~merge ~updates
    main =
  (* main event at T held until updates-watermark >= T, then
     enriched with as_of(key, T) *)
```

Correctness under *late* updates is the whole point, and it rests on watermarks exactly as elsewhere: a main event at time T is *buffered* until the updates stream's watermark reaches T. Only then are all updates with $valid\_from \leq T$ guaranteed to have arrived — including ones that were delayed in transit — so `as_of` returns the correct version. This converts the arrival race into a deterministic event-time result: the output is identical regardless of the order in which rows and updates physically arrive. The cost is the usual event-time trade-off — buffering adds latency and memory, and a silent updates stream stalls the watermark (addressed separately by idle-watermarks). The guarantee is only as strong as the updates watermark's promise that no earlier-stamped update is still coming.

7 RETRACTIONS FOR LATE DATA

When a late `Data` arrives after a window has fired, the operator emits:

$$\texttt{Retract}(v_{\text{old}},\, t) \;\rightarrow\; \texttt{Data}(v_{\text{new}},\, t)$$

Two aggregator strategies:

INVERTIBLE `remove`$(\texttt{acc}, x)$ is defined. Late data: $O(1)$ update via `add`/`remove`. Examples: sum, count, average.

RECOMPUTE No inverse. Late data triggers full recomputation over the buffered event list: $O(n)$. Examples: max, min, percentile.

Both strategies produce identical external behaviour: the downstream sees a `Retract`/`Data` pair and can perform an upsert.

8 ORTHOGONAL RUNTIME

Each production capability has three files: `foo.mli` (contract), `foo_noop.ml` (zero-overhead stub), `foo_default.ml` (production implementation). `runtime.ml` assembles them by `mode`:

```
type mode = Noop | Log | Parallel | Prod

(* Noop: zero overhead, for tests *)
(* Log:  stderr metrics + SIGTERM handler *)
(* Prod: Prometheus HTTP :9090 + shutdown + DLQ log *)
```

Dead Letter Queue. `safe_decode` wraps every codec call; decode errors send the raw bytes to the DLQ with topic, error message, and timestamp instead of crashing the pipeline.

Graceful Shutdown. A `guarded_source` checks `is_requested()` before each poll. On SIGTERM: source returns `None`, pipeline drains its buffer, sink flushes, process exits.

Prometheus Metrics. `metrics_log` implements counter, gauge, and histogram (7 buckets, $1\mu s$–$1s$) with thread-safe `Mutex`-protected state and a TCP HTTP server that serves the exposition text format on `GET /metrics`.

9 PARALLELISM

Data parallelism via key sharding: `hash(key) mod N` routes each event to a dedicated worker. Workers share no state — window correctness follows from the invariant that all events for a given key go to the same shard. The hash uses `land max_int` (not `abs`, which returns `min_int` for `min_int`) to guarantee a non-negative shard index.

Crash resilience. A worker body is wrapped in `try/with`; on an exception it marks itself failed and closes its input channel. The dispatcher uses `try_push` (returning `false` on a closed channel) and skips failed shards, so a single worker crash can no longer deadlock the whole pipeline by blocking the dispatcher on a full channel that nobody drains.

Per-worker sinks. `run_parallel_simple` takes an optional `~sink_factory : int -> 'b -> unit`. When supplied, each worker writes to its own sink (e.g. its own Kafka partition) with zero lock contention; otherwise a single shared sink is guarded by a mutex. This removes the global-mutex bottleneck that flattened throughput at high worker counts.

Table 2. Benchmark: 1M events, 1000 devices, tumbling 30s, OCaml 4, 1 CPU

Mode	ev/s	Time (s)	Speedup
Sequential	209,293	4.78	1.00×
1 worker	151,570	6.60	0.72×
2 workers	405,775	2.46	1.94×
4 workers	519,463	1.93	2.48×
8 workers	599,988	1.67	2.87×

The sub-linear speedup on 1 CPU is expected: OCaml 4's GIL prevents true parallelism, but channel-mediated pipelining hides latency. On OCaml 5 with `Domain`, channel overhead drops from ∼106 ns (Mutex) to ∼30 ns (Atomic), and real 4-core parallelism is expected to yield 3.5–3.8×.

10 PERSISTENT STATE

The `state_backend` contract (`get/set/delete/snapshot/restore`) has three implementations: `_noop`, `_memory` (Hashtbl), and `_rocksdb` backed by RocksDB through a C FFI. The FFI binds `rocksdb_open/put/get/delete` from the RocksDB C API. GC safety is maintained by copying all bytes across the OCaml↔C boundary immediately — the `get` stub `memcpy`s the returned buffer into a fresh OCaml string and frees the C buffer before returning, so no OCaml pointer is ever retained on the C side. The `rocksdb_t` handle lives in a custom block whose finalizer closes the database. State persists across process restarts, verified by a test that writes, closes, reopens the same path, and reads the values back.

11 TESTING

Harness. A deterministic test context `('a,'b) t` feeds events via `push`/`push_wm` and collects output via `run`. No broker, no threads, no wall clock.

Thirteen test suites run under `dune test`. Beyond the operator-correctness unit tests and QCheck property tests, the suite targets the failure modes specific to stream engines:

Determinism replay determinism (same input twice yields byte-identical output); parallel result equals sequential as a *multiset* (order between keys is nondeterministic by design, so multisets are compared — this proves key-sharding neither drops nor

duplicates); watermark fuzzing on randomly time-shuffled streams; permutation invariance of order-independent aggregates.

Exactly-once parallel a checkpoint commits one snapshot per worker; state is restored after a simulated restart; exactly n outputs for n inputs.

Crash resilience a watchdog-timed test crashes one worker and asserts the others still finish — without the fix it hangs.

Soak a separate slow target runs millions of events and asserts live heap growth $\leq 3\times$ (a leak would be $\sim 10\times$), validating dedup and window eviction.

Property tests (QCheck, 100–500 cases each):

(1) $\sum_w \texttt{count}(w) = |\texttt{input}|$ — window preserves all events.
(2) $\texttt{dedup}(\texttt{dedup}(s)) = \texttt{dedup}(s)$ — dedup is idempotent.
(3) Watermarks are monotone for any event order.
(4) `enrich` preserves event count (left join).
(5) `critical` $\Leftrightarrow$ `speed` $> 1.3 \times$ `limit` — for all `float` pairs.
(6) `map` preserves event count.
(7) $|\texttt{filter}(s)| \leq |s|$.

12 CONCLUSION

miniFlink demonstrates that the core semantics of Apache Flink — event time, watermarks, windowing (five types including session with merging and a trigger-driven global window), stateful operators, exactly-once (single-thread and parallel via barrier snapshots, with recovery and a transactional sink), table/join with TTL eviction, retractions, and production layers (DLQ with retry, graceful shutdown, Prometheus metrics, structured logging, health/config, RocksDB-backed state) — fit in ∼3000 lines of OCaml across 27 test suites. Two design decisions proved most consequential:

(1) The `KEYED` type class eliminates repeated key extraction, making the pipeline read as a specification.
(2) The `(type a)` locally abstract type annotation in `Pipe.window` avoids the recursive type $\alpha = \texttt{string} \times \alpha\ \texttt{list}$ that otherwise requires `Obj.magic`.

The OCaml module system's ability to accept modules as runtime values (first-class modules) is what makes both design decisions expressible without metaprogramming or code generation.

Deliberate non-goals keep the "mini" honest: distributed execution, remote shuffle, end-to-end exactly-once across an external source/sink, and incremental checkpointing are out of scope. The project targets the *semantics* of stream processing on a single node, not the distributed infrastructure of a production cluster.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.